

PROJETO KENZIE 360

Arquitetura Lambda na Google Cloud Platform

Estado atual da implementação, arquitetura completa e roadmap

Documento de apresentação — Sprint 2
Bot de atendimento V2 — dados majoritariamente CDE
Agosto de 2026

1. Objetivo deste documento

Este documento consolida a arquitetura de dados do Projeto Kenzie 360, reunindo três fontes que foram produzidas em momentos diferentes do trabalho:

- O desenho lógico da arquitetura Lambda (Sprint 1), mapeando as camadas Batch, Speed e Serving ao modelo de Data Lake RAW → TRUSTED → CURATED;
- O briefing acadêmico em PDF, com a mesma proposta em formato resumido;
- O repositório de infraestrutura como código (Terraform + GitHub Actions), que é a parte do projeto que já foi efetivamente implementada e provisionada no GCP.

O objetivo é apresentar a arquitetura completa de forma honesta quanto ao estado atual: o que já está rodando de verdade, o que existe apenas como desenho conceitual, e o que ainda depende de decisões futuras do time. Isso evita a leitura de que "o projeto está pronto" quando, na prática, a Sprint 2 entregou a fundação — os depósitos de dados e a esteira de automação — e não a esteira de dados fim a fim.

Nota de revisão de 01/09/2026. A primeira versão deste documento auditou o repositório de infraestrutura e concluiu, corretamente para aquele escopo, que o processamento entre as camadas e o mascaramento de PII não estavam implementados. Uma verificação posterior mostrou que ambos estão em operação há mais de uma semana, com 36.000 atendimentos e 405.073 mensagens carregados. A divergência não era de leitura: é que o código da pipeline vive apenas na pasta local do projeto e nunca foi versionado no repositório.

Esse episódio é o argumento mais concreto a favor do item de versionamento listado na seção 7. Um integrante da própria equipe, auditando o repositório com método correto, concluiu que a camada de dados não existia. Um avaliador externo chegaria à mesma conclusão mais rápido. As linhas afetadas da tabela da seção 2 e da tabela da seção 6 foram corrigidas.

2. Estado atual do projeto

A tabela abaixo resume o que existe hoje, com evidência de onde isso está implementado, contra o que ainda é apenas planejado ou conceitual.

Componente	Status	Evidência
Bucket RAW no Cloud Storage	Implementado e adotado via Terraform	infra/modules/data-lake/main.tf — google_storage_bucket.raw
Datasets BRONZE / SILVER / GOLD no BigQuery	Implementado e adotado via Terraform	infra/modules/data-lake/main.tf — google_bigquery_dataset
Pipeline de CI/CD (validação + plano + apply)	Implementado	.github/workflows/infra-cicd.yml
Ambiente dev	Provisionado e adotado (terraform import)	infra/environments/dev/terraform.tfvars
Ambiente prod	Estrutura pronta, projeto GCP ainda não existe	infra/environments/prod/terraform.tfvars (placeholder)
Processamento Batch (Dataflow/SQL) que move RAW→BRONZE→SILVER→GOLD	Implementado e em operação	01_carga_bronze.py · 02_cria_silver.py · 06_cria_gold.py · 07_cria_star_schema.py e os DDLs em sql/ — 36.000 atendimentos e 405.073 mensagens carregados
Speed layer (Pub/Sub streaming)	Conceitual. Passou a ter função definida: serve a decisão de	Rascunho_Arquitetura_Lambda_GCP.md, nota didática

Componente	Status	Evidência
	roteamento em tempo real	
Mascaramento de PII / DLP	Implementado em SQL; DLP não utilizado	02_cria_silver.py — regex de CPF e e-mail na transição bronze→silver, com a coluna pii_remascarada marcando as linhas afetadas
Módulo de IAM granular por camada	Planejado, não implementado	infra/README.md, "próximos incrementos"
Orquestração entre camadas	Decidido: Cloud Scheduler + Cloud Run Jobs. A construir	infra/README.md — Composer não tem free tier (~US\$525/mês)
Pipeline de CI/CD da aplicação (chat)	Desenhado, mas fora do ar	docs/proposta-cicd-aplicacao.yml, movido para fora de .github/workflows

3. Arquitetura de dados completa

A arquitetura combina o desenho conceitual da Lambda Architecture (Batch, Speed e Serving) com a implementação real em Terraform, que usa uma nomenclatura mais granular — o padrão "medalhão" (RAW → BRONZE → SILVER → GOLD). O BRONZE é a camada que aparece no código mas não estava descrita nos documentos de briefing: é o ponto em que o CSV bruto vira, pela primeira vez, uma tabela SQL consultável, antes de qualquer limpeza.

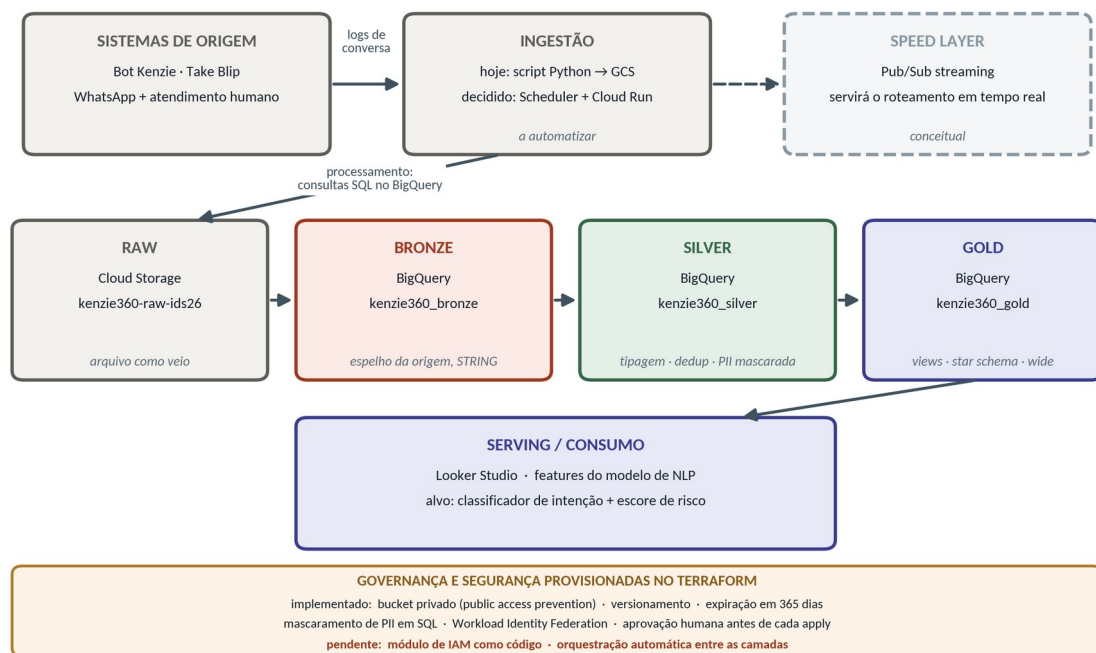


Figura 1 — Fluxo de dados desde a origem até o consumo, com o estado de cada etapa.

3.1 O ciclo de aprendizado: o caminho de retorno

A Figura 1 descreve corretamente o fluxo do dado, mas é um fluxo de mão única: o dado entra pela ingestão, atravessa as camadas e termina no consumo. Um pipeline assim produz relatórios. O que o projeto se propõe a construir é um sistema que melhora com o uso, e isso exige um caminho de retorno que a Figura 1 não contém.

A Figura 2 fecha esse ciclo. Conversas novas geram rotulagem, que alimenta o retreino, que produz um modelo melhor, que decide o roteamento do próximo atendimento — e esse atendimento vira conversa nova. Cada

estágio traz os serviços que o executam e declara sua própria situação: implementado, a construir nesta entrega, ou apenas desenhado.

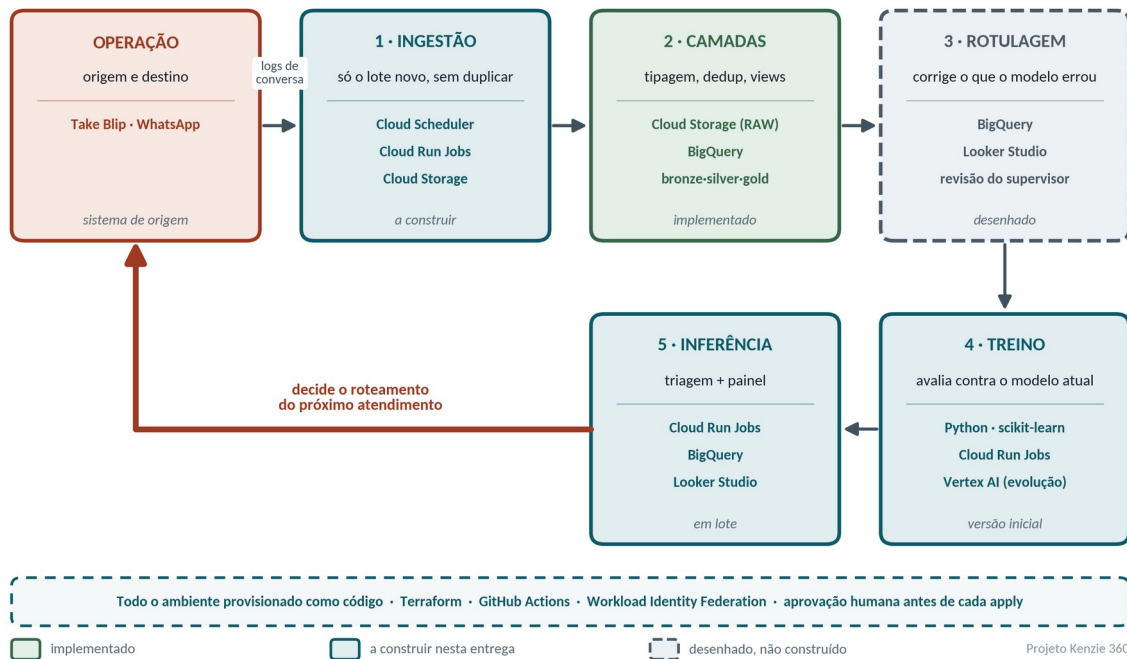


Figura 2 — O ciclo de aprendizado, com os serviços de cada estágio. A seta em destaque é o caminho de retorno que distingue um sistema que aprende de um pipeline analítico.

O estágio de rotulagem é o que não constava de nenhum desenho anterior, e é a peça de maior valor para a aplicação em operação real: é ele que transforma a correção do supervisor em exemplo de treino. Sem esse estágio, o modelo é treinado uma vez e degrada em silêncio.

3.2 Ferramentas da camada de dados e por que foram escolhidas

Ferramenta / recurso	Papel na arquitetura	Por que foi escolhida
Cloud Storage (bucket RAW)	Armazena o dado exatamente como chega, sem tratamento	Barato para grandes volumes de arquivo bruto; não exige schema na entrada; suporta versionamento e expiração automática
Cloud Functions + Pub/Sub	Ingestão do evento de conversa	Pub/Sub desacopla quem gera o dado de quem consome; permite tanto o trilha batch quanto, futuramente, o trilha streaming a partir do mesmo evento
BigQuery (BRONZE)	Primeira materialização do CSV como tabela, com colunas de rastreabilidade	Data warehouse serverless: não há servidor para dimensionar e o custo escala com o uso
BigQuery (SILVER/trusted)	Limpeza, deduplicação e tratamento de PII	Permanece no BigQuery porque a transformação é feita via SQL/Dataflow direto sobre as tabelas, sem mover o dado entre sistemas
BigQuery (GOLD/curated)	Dados agregados, prontos para dashboard e features de NLP	Mesma razão do SILVER — e joins entre datasets só funcionam se estiverem na mesma região, o que reforça manter tudo no BigQuery
Dataflow (planejado)	Motor de transformação batch e	É o serviço gerenciado do GCP para pipelines

Ferramenta / recurso	Papel na arquitetura	Por que foi escolhida
	streaming entre as camadas	Apache Beam, cobrindo tanto o trilha batch quanto o speed com a mesma tecnologia
Looker Studio	Visualização dos KPIs para a diretoria e áreas de negócio	Conecta nativamente ao BigQuery sem custo adicional de licença
Região us-central1	Localização de todos os recursos	Única região com Always Free no Cloud Storage; e o BigQuery não executa joins entre datasets em regiões diferentes — a consulta falha

4. Infraestrutura como código e pipeline de CI/CD

Esta é a parte do projeto mais madura tecnicamente até o momento: o data lake não foi criado clicando no console do GCP, e sim descrito como código (Terraform), versionado no Git e aplicado por um pipeline automatizado no GitHub Actions.

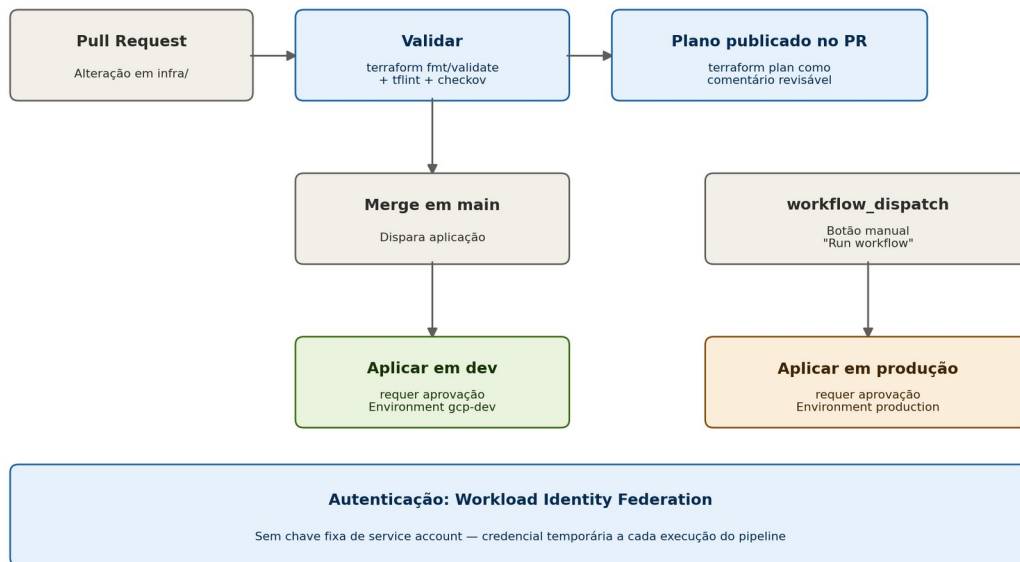


Figura 3 — Fluxo do pipeline de CI/CD: validação automática, plano revisável e aprovação humana antes de qualquer aplicação.

4.1 Ferramentas do pipeline e por que foram escolhidas

Ferramenta	Função	Por que foi escolhida
Terraform	Descreve a infraestrutura como código	Torna a configuração auditável (histórico no Git), reproduzível e revisável antes de tocar em dados reais
terraform import	Faz o Terraform "adotar" recursos já criados manualmente na Trilha 0	Evita que o Terraform tente recriar (e duplicar) o bucket e os datasets que já têm dados
tfint	Lint de boas práticas específicas do provedor GCP	Pega erro de configuração (tipo de recurso obsoleto, atributo incorreto) antes de chegar ao ambiente real
checkov	Varredura de segurança (IaC security scanning)	Detecta bucket público, IAM permissivo e outras falhas de segurança já no Pull Request, em modo

Ferramenta	Função	Por que foi escolhida
		soft-fail até o time zerar os achados
GitHub Actions	Orquestra validação, plano e aplicação	Já integrado ao repositório; permite publicar o plano do Terraform como comentário no próprio PR, sem precisar abrir a aba de logs
Workload Identity Federation (WIF)	Autenticação do pipeline no GCP	Elimina a necessidade de uma chave de service account fixa — o vetor de vazamento mais comum em CI/CD — gerando credencial temporária a cada execução
GitHub Environments (gcp-dev / production)	Porta de aprovação manual antes de aplicar	Garante que nenhuma mudança é aplicada de forma irreversível sem uma pessoa revisar e aprovar
Separação dev / prod	Isola o ambiente de teste do ambiente real	Qualquer erro de configuração aparece primeiro no ambiente de teste, sem risco a dados reais de correntistas

5. Governança e segurança

5.1 Já implementado no Terraform

- Bucket RAW com acesso público bloqueado por padrão (`public_access_prevention = enforced`);
- Uniform bucket-level access, para o controle de permissão ser consistente em todo o bucket;
- Versionamento do bucket RAW, para recuperar um arquivo sobrescrito por engano;
- Expiração automática dos dados crus após 365 dias (`raw_retention_days`), limitando por quanto tempo dado sensível fica retido;
- `force_destroy = false` no bucket RAW, exigindo um passo extra para apagar o bucket, evitando exclusão acidental;
- Rótulos (labels) de projeto e ambiente em todos os recursos, para rastreabilidade de custo e responsabilidade;
- Autenticação do pipeline sem chave fixa (Workload Identity Federation);
- Aprovação humana obrigatória antes de aplicar em dev e em produção.

5.2 Planejado, ainda não implementado

- Mascaramento de PII / integração com Cloud DLP antes de dados reais de cliente entrarem na camada SILVER;
- Módulo de IAM versionado como código, definindo quem lê/escreve em cada camada;
- Orquestração automática entre as camadas com Cloud Scheduler + Cloud Run Jobs, decisão já tomada (o Cloud Composer foi descartado por custo, cerca de US\$ 525/mês, sem tier gratuito);
- Módulo de Vertex AI. O alvo do modelo foi definido em 28/08: pipeline de dois estágios, com classificador de intenção e escore de risco.

6. Pontos de atenção entre os documentos do projeto

Ao cruzar o briefing (markdown e PDF) com o código real do repositório, três divergências merecem uma decisão do time antes da entrega final:

Ponto	O que o briefing diz	O que o código mostra
Mascaramento de PII	"O briefing está correto: o mascaramento é aplicado"	O README da infra lista o DLP

Ponto	O que o briefing diz	O que o código mostra
	na transição bronze→silver" (markdown, seção de governança)	como próximo incremento, mas o mascaramento por SQL já opera. A divergência é de local: o código da pipeline não está versionado no repositório de infraestrutura
Camada BRONZE	Não mencionada — o briefing descreve apenas RAW → TRUSTED → CURATED	Existe como camada intermediária explícita no Terraform (kenzie360_bronze)
Nº de colunas das tabelas	30 colunas (conversas) / 12 (mensagens)	Resolvido: 30 e 12 são os valores corretos da base v8. Os números 23 e 9 vinham do gerador v5/v6 e foram corrigidos ainda na Sprint 1

Recomenda-se que a versão final da documentação escolha uma narrativa única sobre PII ("mascaramento planejado para a Sprint 3", por exemplo, em vez de "já implementado") e decida se o BRONZE entra explicitamente no glossário do trabalho.

7. Próximos passos (roadmap)

Na mesma estrutura de módulos que infra/modules/data-lake/ já estabeleceu, os próximos incrementos planejados são:

- Módulo de IAM — permissões de leitura/escrita por camada, versionado como código;
- Orquestração — mover dados entre RAW, BRONZE, SILVER e GOLD automaticamente, quando os DAGs/jobs forem escritos;
- Vertex AI — quando o alvo do modelo de NLP for definido;
- DLP / mascaramento de PII — antes de qualquer dado real de cliente circular pela camada SILVER;
- Esteira de CI/CD da aplicação do chat — já desenhada em docs/proposta-cicd-aplicacao.yml, aguardando o código da aplicação (app/, Dockerfile, testes) entrar no repositório.

Versionar o código da pipeline de dados (scripts Python e DDLs) em repositório próprio, estendendo a ele o padrão de promoção por Pull Request já em operação na infraestrutura. É a maior lacuna atual e a causa da divergência descrita na nota de revisão

8. Fontes deste documento

- Rascunho_Arquitetura_Lambda_GCP.md — desenho lógico da Sprint 1
- Arquitetura_Lambda_GCP_-_Projeto_Kenzie_360_FINAL.pdf — briefing acadêmico
- Repositório MACK-Hand-on/MACK (infra/, .github/workflows/) — infraestrutura como código e pipeline de CI/CD, estado real em agosto de 2026